

# MODULARIZAÇÃO EM C: DO CAOS À ARQUITETURA LIMPA

Procedimentos, Funções e a Engenharia de Software

Curso Técnico em Desenvolvimento de Sistemas | Semana 7



# O PROJETO DA AULA (OBJETIVOS)



Organizar código em funções coesas.



Diferenciar funções com retorno e procedimentos (void).



Aplicar passagem por valor e por referência.



Compreender o Tipo Abstrato de Dados (TAD)



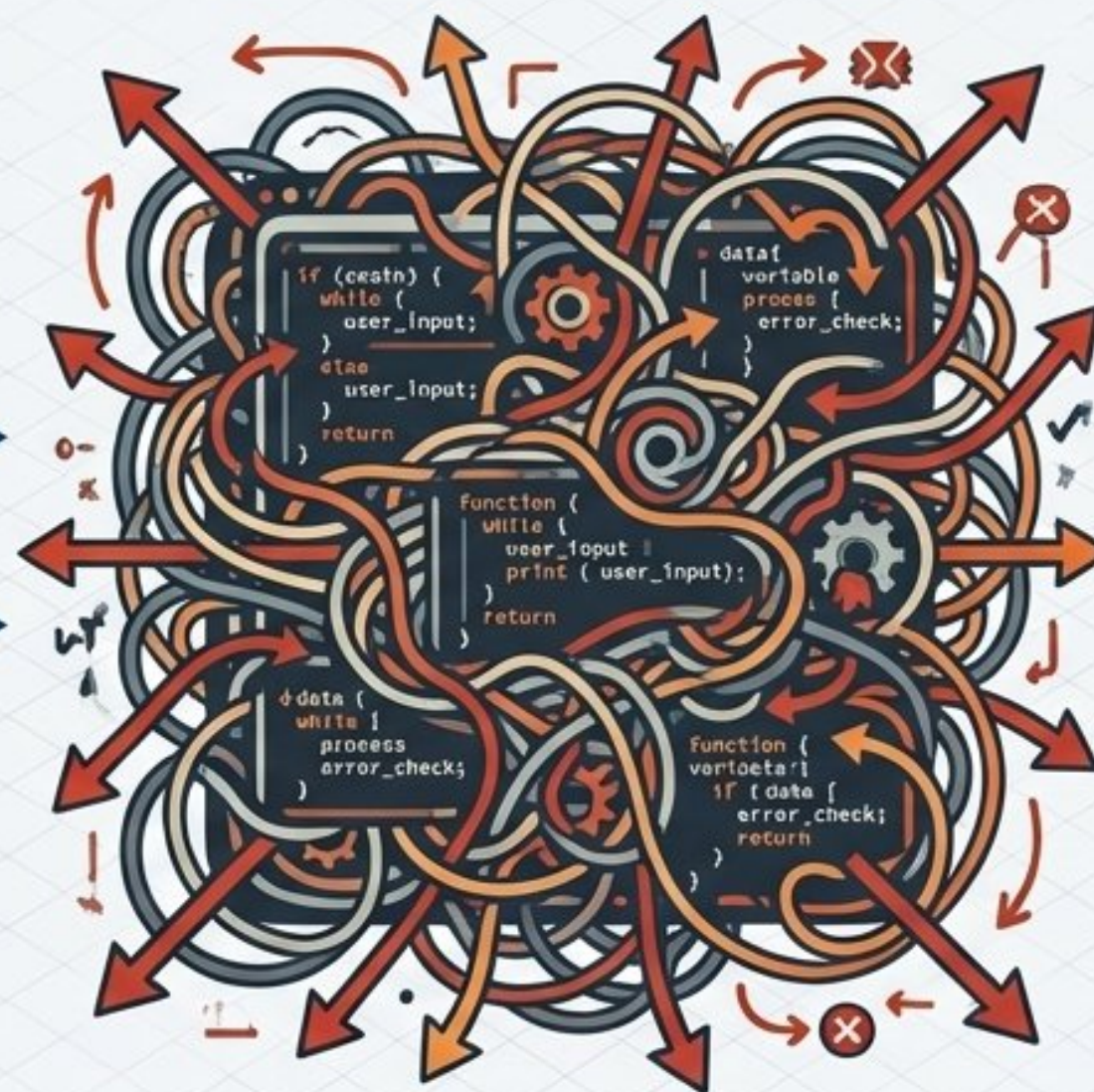
# O PROBLEMA DO CÓDIGO MONOLÍTICO

! Tudo dentro da main().

! Dificuldade extrema de leitura.

! Alto risco de quebra na manutenção.

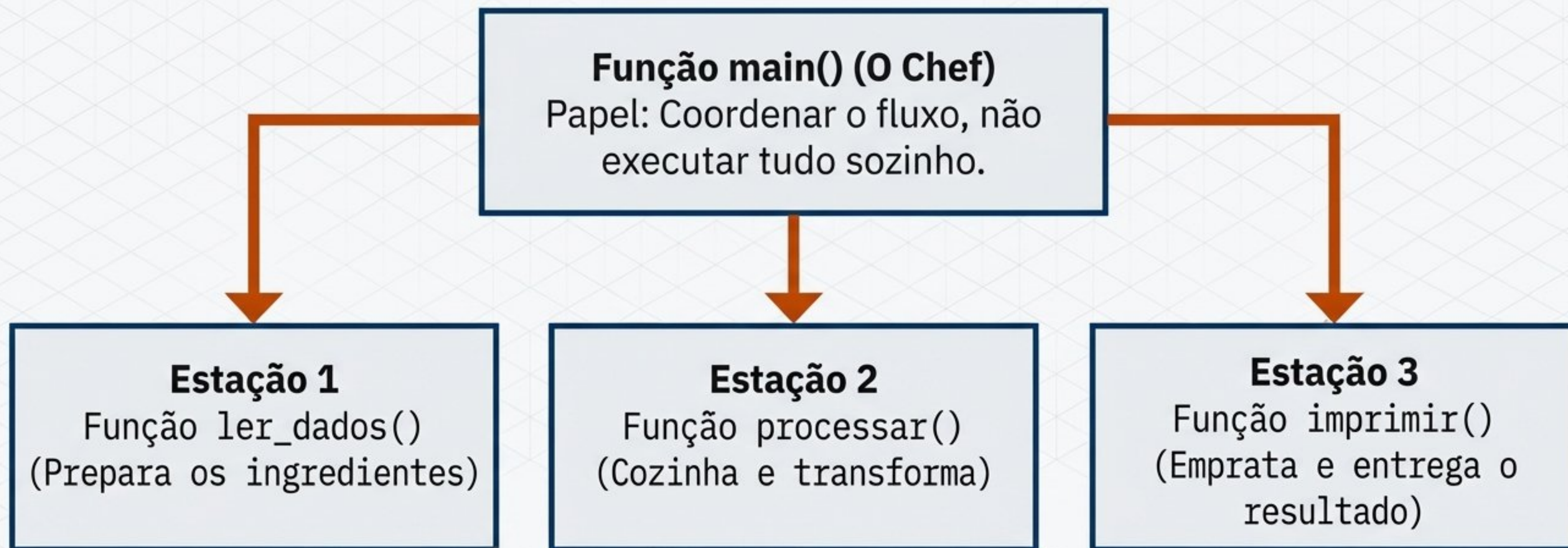
! Impossível dividir o trabalho em equipe.



Se o seu código parece um **prato de espaguete**, é hora de **modularizar**.



# A FILOSOFIA DA MODULARIZAÇÃO: A COZINHA PROFISSIONAL



Vantagens: Legibilidade, Manutenção Isolada, Reuso e Organização de Equipe.



# A ANATOMIA DE UMA FUNÇÃO EM C

## Tipo de Retorno

O que a função devolve.

## Assinatura/Nome

Identificador claro da ação.

```
float calcular_media(float n1, float n2) {  
    float resultado = (n1 + n2) / 2;  
    return resultado;  
}
```

## Parâmetros

Os dados de entrada necessários.

## Corpo / Escopo

Onde a mágica acontece de forma isolada.

## Retorno

A entrega do resultado ao chamador.

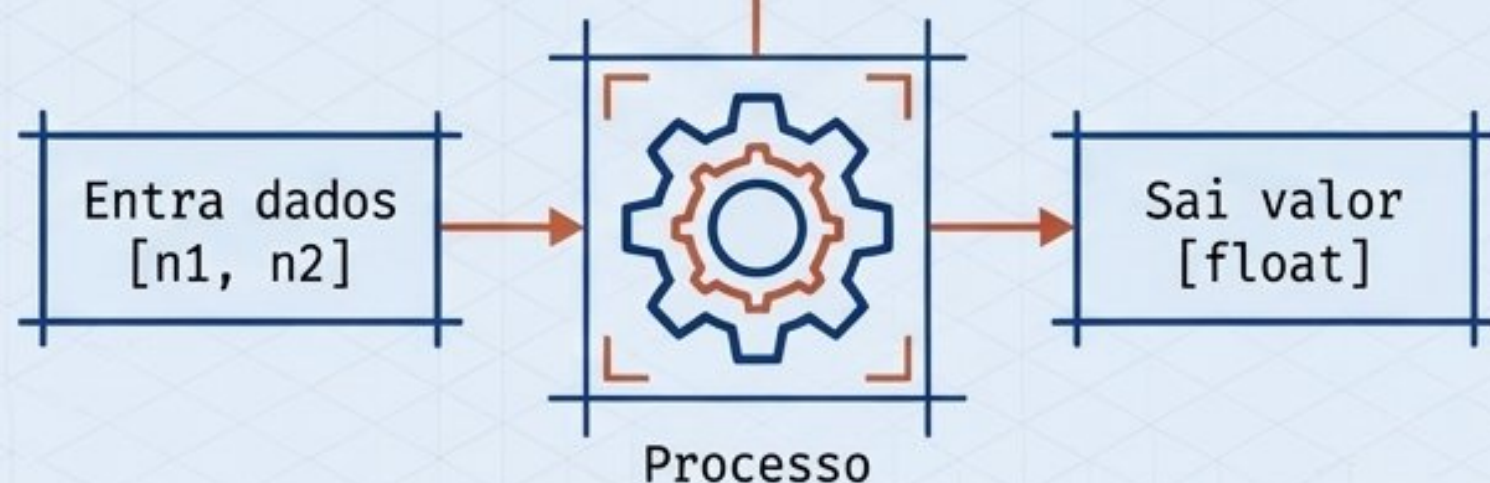


# FUNÇÕES VS. PROCEDIMENTOS

## Funções com Retorno

**Conceito:** Calcula e devolve um valor ao sistema.

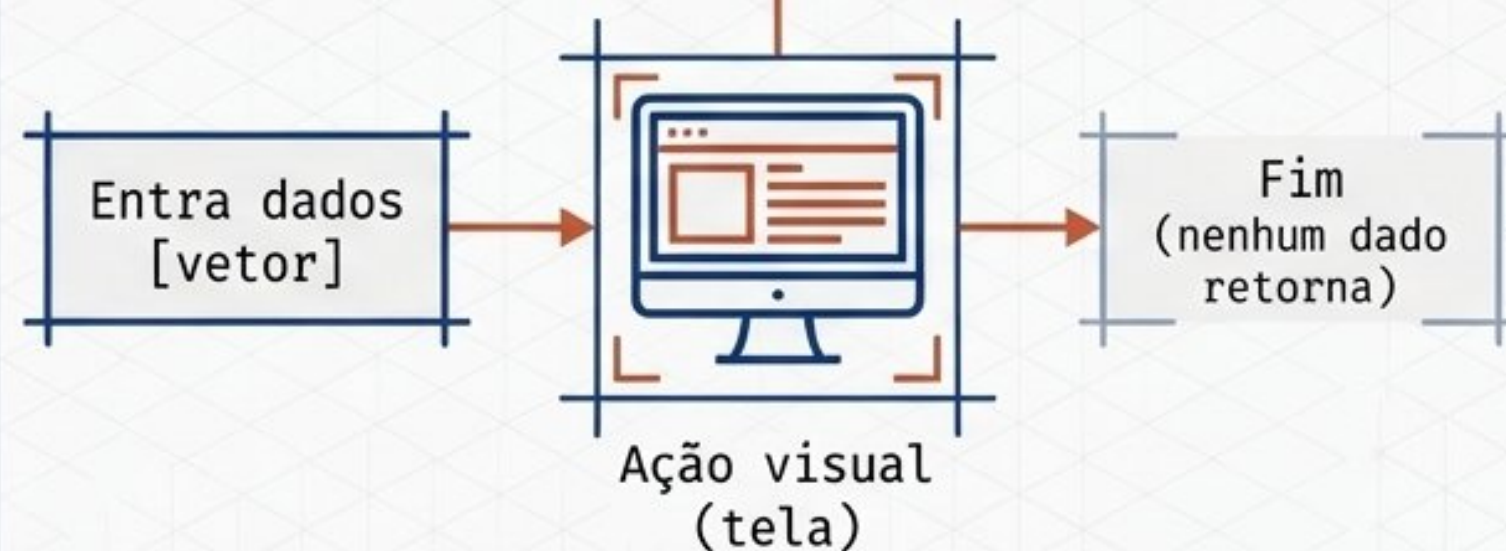
```
float media(float n1, float n2)
```



## Procedimentos (void)

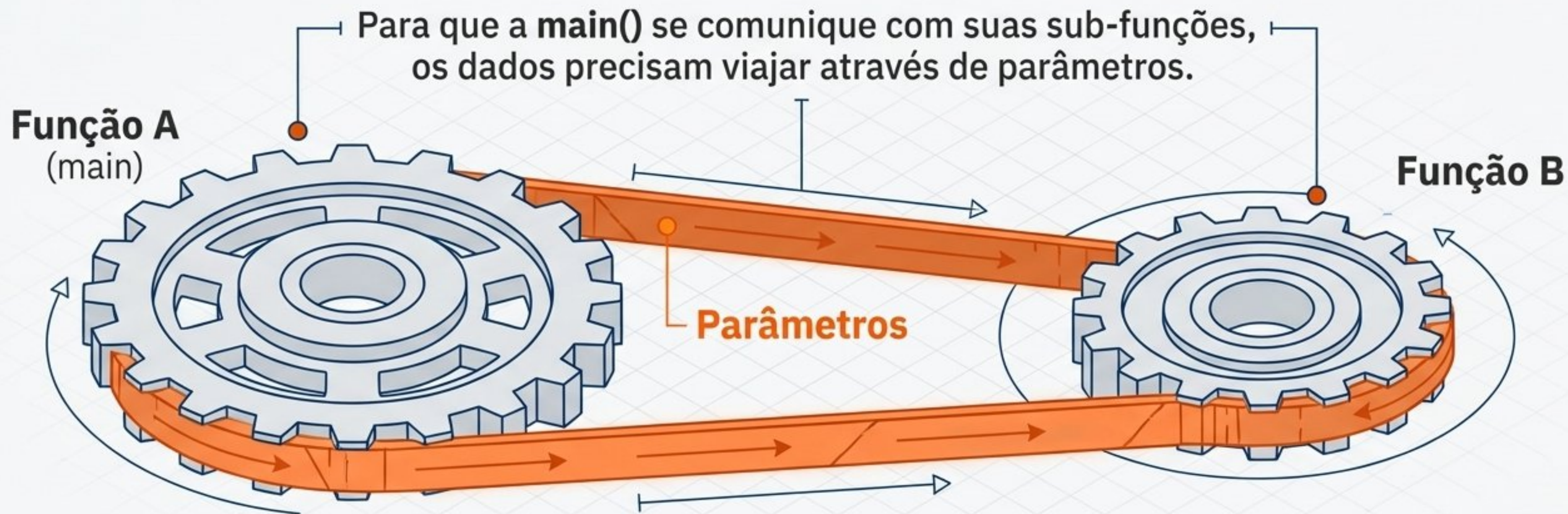
**Conceito:** Executa uma ação, mas não devolve nenhum dado.

```
void imprimir_relatorio(int v[], int n)
```





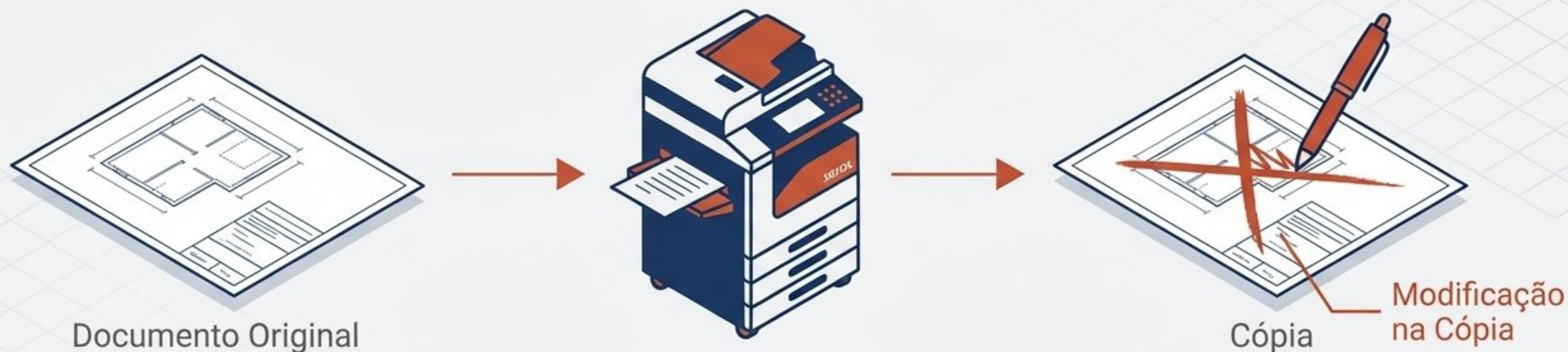
Variáveis dentro de uma função são 'invisíveis' para o resto do programa (Escopo Local).



Existem duas formas dessa viagem acontecer: enviando uma cópia ou enviando o mapa do endereço original.



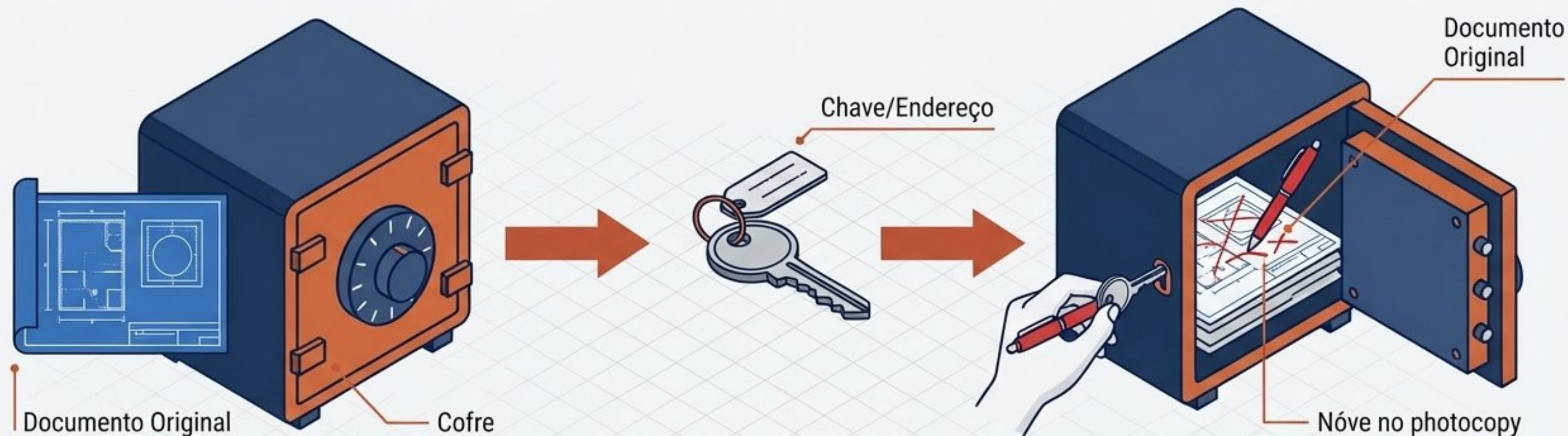
# PASSAGEM POR VALOR (A FOTOCÓPIA)



| Mecanismo:                                          | Regra de Ouro:                                                                            | Sintaxe:                                                         | Uso Ideal:                                                            |
|-----------------------------------------------------|-------------------------------------------------------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------|
| A função recebe apenas uma cópia do valor numérico. | Alterações feitas dentro da função <b>não afetam</b> a variável <b>original externa</b> . | <pre>void calcular(int x)            /         Cópia Local</pre> | Quando precisamos apenas ler ou usar um dado para um cálculo isolado. |



# PASSAGEM POR REFERÊNCIA (O ENDEREÇO ORIGINAL)



## Mecanismo:

A função recebe o endereço de memória da variável (usando ponteiros \* e &).

## Regra de Ouro:

A função altera o dado diretamente na origem. Não há cópias.

## Sintaxe:

Chamada: **alterar(&x)** |  
Declaração: **void alterar(int \*p)**

## Uso Ideal:

Quando uma função precisa atualizar variáveis da main ou manipular grandes estruturas sem duplicar memória.

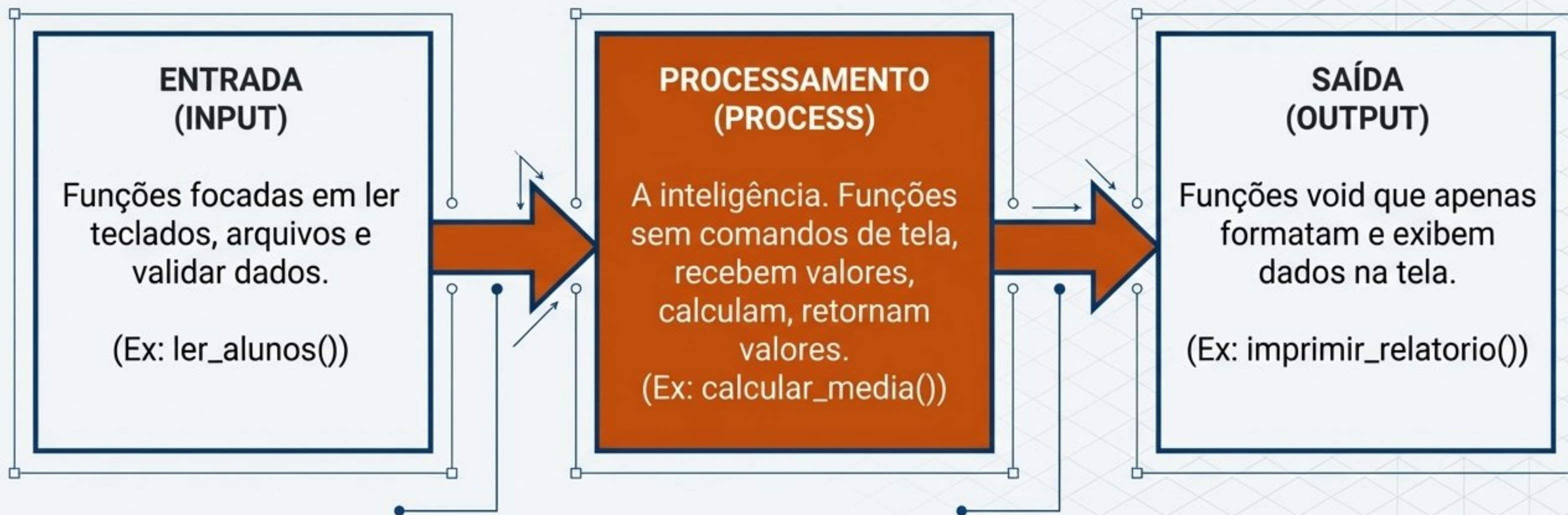


# MATRIZ DE DECISÃO: VALOR VS. REFERÊNCIA

|                    | VALOR                           | REFERÊNCIA                                                  |
|--------------------|---------------------------------|-------------------------------------------------------------|
| Mecanismo          | Cópia do dado                   | <u>Endereço do dado</u> →                                   |
| Impacto na Memória | Aloca nova memória para a cópia | Nenhuma duplicação (usa o mesmo espaço)                     |
| Efeito Colateral   | Variável original protegida     | <u>Variável original é modificada diretamente</u> →         |
| Sintaxe em C       | <code>func(int x)</code>        | <code>func(int *p)</code> e chamada com <code>&amp;x</code> |



# ARQUITETURA DO PROGRAMA: SEPARAÇÃO DE RESPONSABILIDADES

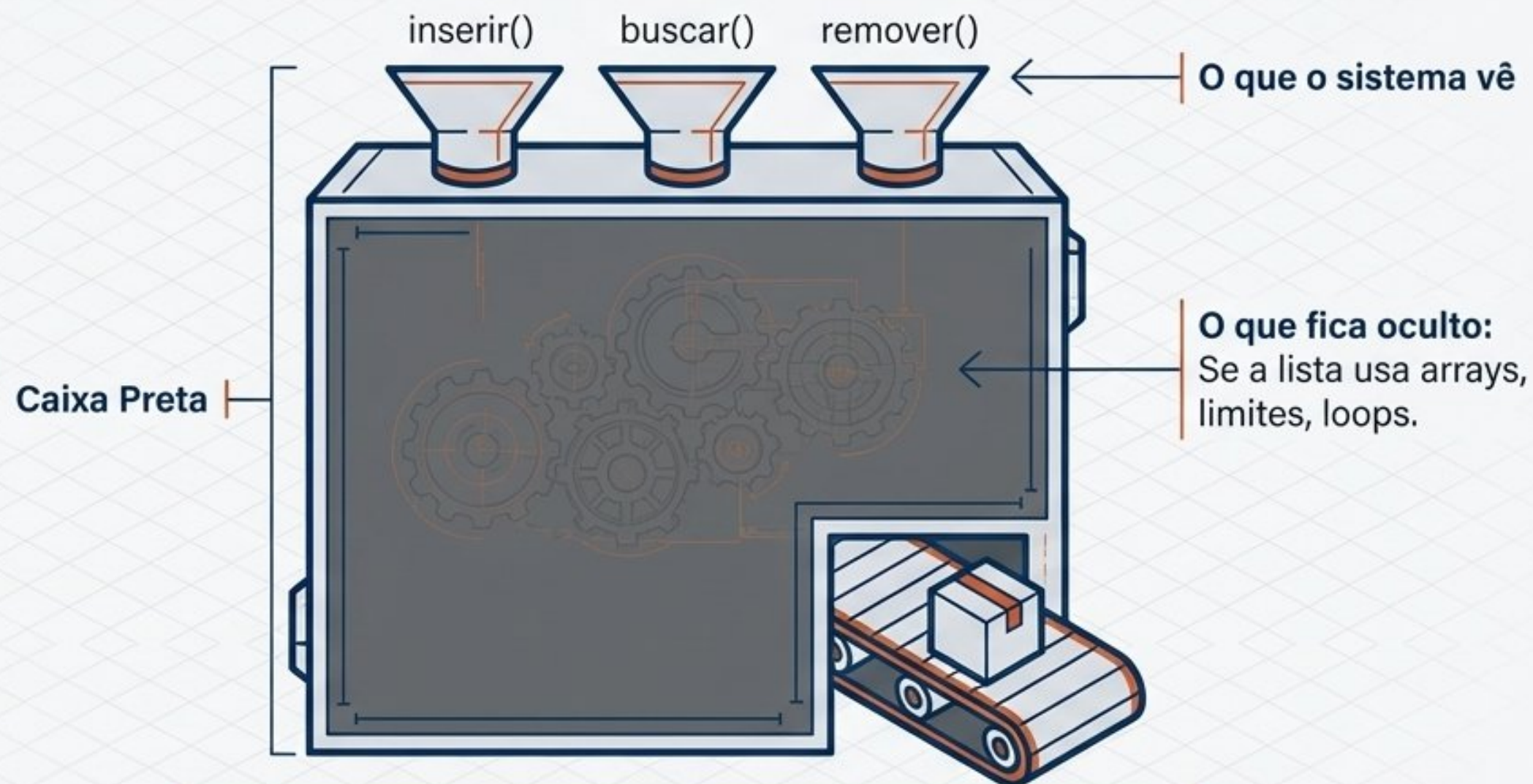


**Nota:** Mantenha os módulos isolados. O processamento nunca deve imprimir dados diretamente!



# INTRODUÇÃO AO TIPO ABSTRATO DE DADOS (TAD)

Um TAD define quais operações podem ser feitas, mas oculta como elas funcionam por dentro.



O Objetivo: Criar sistemas onde a substituição de uma engrenagem interna não quebra a máquina inteira.



# O IMPACTO DA ARQUITETURA: ESTUDO DE CASO

## Antes: O Monólito

```
int main() {
    ler_alunos(turma, n);
    calcular_medias(turma, n);
    imprimir_relatorio(turma, n);
    return 0;

    // ... código ...
    turma = ler_alunos(turma, n);
    calcular_medias(turma, n);
    imprimir_relatorio(turma, n);
    // ... código ...
    return 0;
}
```

Diagram illustrating the monolithic code structure. The code is a single block with various functions and logic mixed together. Annotations with arrows point to different parts of the code, highlighting its complexity and lack of modularity.

**Status:** Confuso e frágil. Lógica de vetores, structs, loops e printf's amontoadas em 100 linhas.

## Depois: Código Modularizado

```
int main() {
    ler_alunos(turma, n);
    calcular_medias(turma, n);
    imprimir_relatorio(turma, n);
    return 0;
}
```

Diagram illustrating the modularized code structure. The code is split into a main function and three separate functions. Arrows indicate the flow of data and control between the main function and the modular functions, showing a cleaner and more organized architecture.

**Status:** A main() age como um painel de controle limpo.



# MISSÃO DE LABORATÓRIO: REFATORAÇÃO



## Prancheta de Tarefas

**Tarefa:** Pegue seu programa da semana anterior (Cadastro de Alunos com Struct e Vetor) que estava em um único bloco monolítico.

- ☐ **Etapa 1:** Crie uma função dedicada void para realizar a leitura de dados.
- ☐ **Etapa 2:** Crie uma função com retorno para o cálculo da média.
- ☐ **Etapa 3:** Crie um procedimento de impressão formatada.

**Regra Arquitetural:** A função main() deve conter apenas declarações de variáveis e a chamada limpa dessas 3 funções.



## O Desafio: Entregar o sistema de Alunos (Struct + Vetor) 100% modularizado.

### Requisitos Críticos:

- Passagem de array de structs corretamente.
- Uso intencional e justificado de void vs. tipo de retorno.
- Uso de passagem por referência onde dados estruturais precisarem ser modificados.



**Foco: Coesão e Escopo Limpo**

Nenhuma função deve fazer duas coisas ao mesmo tempo. Sucesso na Arquitetura.